MULTI-FORMAT STEGANALYSIS SYSTEM

A PROJECT REPORT

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING

By
Batch - 2

Gadepalli Sai Praphulla (323103210061)

Gannavarapu Rishita Lakshmi (323103210064)

Ganti Gayathri (323103210066)

Gayatri Misro (323103210068)

Under the esteemed guidance of
Dr. N. Sharmili

Professor

CSE Department

Department of Computer Science and Engineering

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

[Approved by AICTE NEW DELHI, Affiliated to Andhra University, Visakhapatnam]

[Accredited by National Board of Accreditation (NBA) for B.Tech.CSE,ECE & IT – valid from 2019-22 and 2022-25]

[Accredited by National Assessment and Accreditation Council (NAAC) with A Grade-Valid from 2022-2027]

Kommadi,Madhurawada,Visakhapatnam–530048

2023 –2027

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report titled "**Multi-Format Steganalysis System**" is the Bonafide work of following III B.Tech. students from the Department of Computer Science and Engineering, Gayatri Vidya Parishad College of Engineering for Women, affiliated with Andhra University, Visakhapatnam. This project was done **as part of Cryptography and Network Security lab** during the academic year 2025 - 2026 (III Year Semester - 2).

Gadepalli Sai Praphulla (323103210061)
Gannavarapu Rishita Lakshmi (323103210064)
Ganti Gayathri (323103210066)
Gayatri Misro (323103210068)

Signature of the Guide

Dr. N. Sharmili MTech.,Ph.D.
Professor,
Dept. of CSE

Signature of the HOD

Dr. P. V. S. L. Jagadamba MTech, Ph.D
Professor,
Dept. of CSE

External Examiner

ABSTRACT

This project presents a **Multi-format Steganalysis System** designed to hide and detect secret information within different types of digital files. With the rapid growth of digital communication, protecting sensitive data has become essential. Steganography allows users to conceal information inside files such as images and documents, while steganalysis helps in identifying the presence of such hidden data.

The system supports multiple file formats including images (PNG, JPG), text files (TXT), Word documents (DOCX), and PDF files. It uses the **Least Significant Bit (LSB)** technique for embedding data in images and a **marker-based approach** for text-based files. During detection, the system analyzes file content and extracts hidden patterns or markers to determine whether any secret information is present.

The implementation is carried out using Python with libraries such as Pillow, NumPy, python-docx, PyPDF2, and ReportLab, along with Streamlit for the user interface. The system is designed to be simple, efficient, and user-friendly, providing quick encoding and detection results.

Additionally, the system ensures data integrity by maintaining the original appearance of files after encoding, making the hidden information imperceptible to users. The detection mechanism is optimized to accurately distinguish between normal and encoded files, reducing false positives. The project demonstrates practical applications in **cybersecurity, digital forensics, and secure communication systems**, where identifying hidden data plays a critical role.

1.SYSTEM REQUIREMENTS

1.1. Software Requirements

Table1.Software Requirements

Software	Version/ Specification
Python	3.10.12(64-bit)
Visual Studio Code Editor	1.85+
Operating System	Windows 11
Pillow(PIL)	9.5.0+
NumPy	1.24.4
CSS	Standard
Streamlit	1.30+
ReportLab	4.0.0+

Python packages

Python packages we used in our project includes the following table 2.

We used the command **"pip install package_name"** to download the packages.

Table2.Python packages

S.No.	PackageUsed	Description
1.	NumPy	Used for numerical computations and handling multi-dimensional arrays efficiently.
2.	Pillow(PIL)	Python Imaging Library used for opening, processing, and saving image files.
3.	Python-docx	Library for creating, reading, and modifying Microsoft Word (.docx) documents.
4.	ReportLab	Toolkit for generating and writing PDF documents programmatically.
5.	Streamlit	Framework used to build interactive web applications for the user interface.

1.2. Hardware Requirements

The below Table 3 shows the minimum hardware requirements of our project.

Minimum Hardware Requirements:

Table 3. Minimum Hardware Requirements

S.No	Hardware Component	Specification
1.	Storage Device	Minimum 100 GB HDD/SSD for storing files, QR codes, and keys
2.	Processor	64-bit processor (Intel/AMD) for efficient execution of Python and cryptographic operations
3.	RAM	Minimum 4 GB
4.	Internet Connection	Required for downloading libraries, updates, and deployment
5.	Display Unit	Monitor with at least 1366×768 resolution
6.	Input Devices	Keyboard and Mouse for user interaction

2.METHODOLOGY

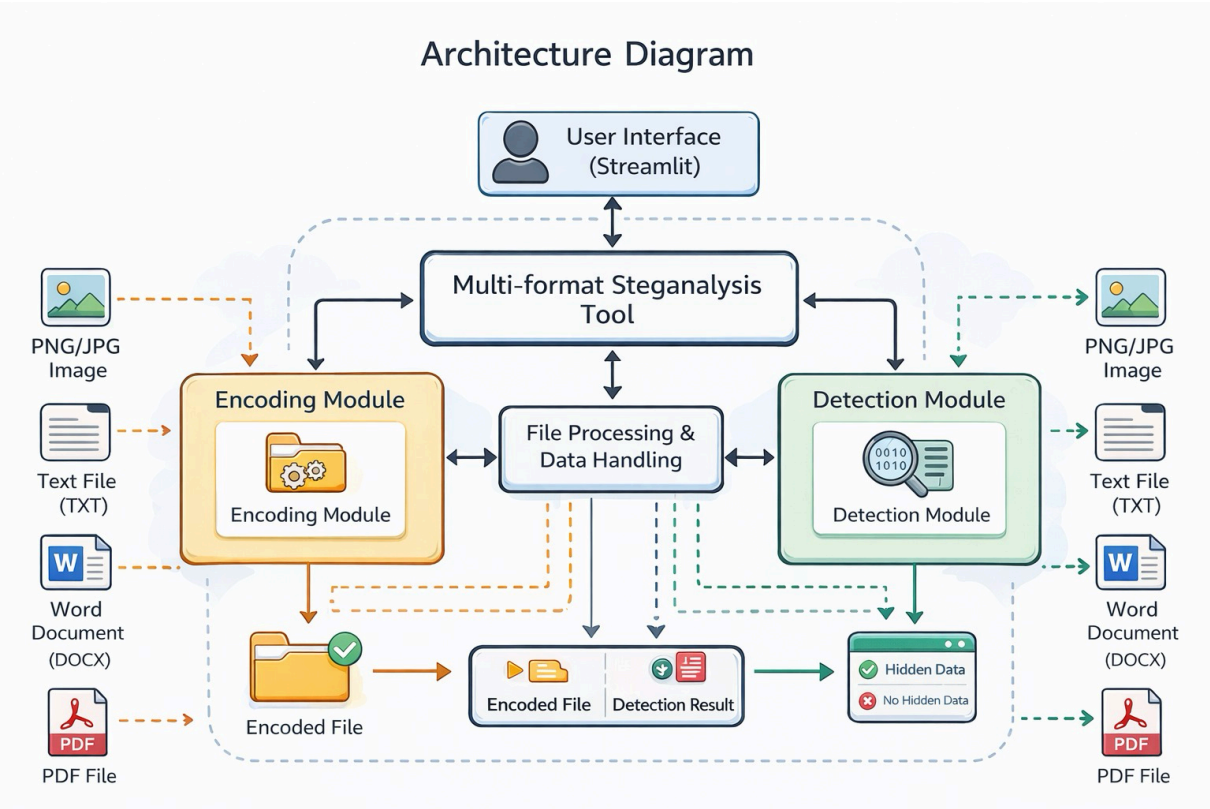
2.1 SYSTEM ARCHITECTURE

The proposed **Multi-format Steganalysis System** follows a client-server architecture implemented using the Streamlit framework. The system is designed as an interactive web-based application where both frontend and backend functionalities are integrated into a single platform, ensuring simplicity, efficiency, and ease of use. The architecture mainly consists of two core modules: the Encoding Module and the Detection Module, supported by file processing and data handling mechanisms.

The **Encoding Module** is responsible for embedding secret messages into various file formats provided by the user. Depending on the type of file uploaded, the system applies appropriate steganographic techniques such as Least Significant Bit (LSB) encoding for images and marker-based embedding for text-based files (TXT, DOCX, PDF). The encoded file is then generated and made available for download.

The **Detection Module** is responsible for analyzing the uploaded file to determine whether hidden data is present. It extracts relevant information from the file and checks for predefined patterns or markers that indicate embedded data. Based on the analysis, the system classifies the file as either containing hidden data or not.

This architecture ensures efficient processing, multi-format compatibility, real-time execution, and a user-friendly interface, making it suitable for applications in cybersecurity and digital forensics.



2.2 DESCRPTION

System Description

The Multi-format Steganalysis Tool is designed to securely embed secret messages within digital files and detect the presence of such hidden data. The system uses steganographic techniques for encoding and simple yet effective detection mechanisms for steganalysis. It is implemented using Python with Streamlit for frontend interaction and various libraries for backend processing. The system operates in two main phases:

1. Encoding Phase

In this phase, the system hides the secret message inside the uploaded file. The process is as follows:

- The user uploads a file through the application interface.
- The user enters a secret message to be hidden
- The system identifies the file type and applies the appropriate encoding method

For Image Files (PNG, JPG, JPEG):

- The image is converted into an RGB format.
- The secret message is converted into binary format.

- The message is embedded into pixel values using the Least Significant Bit (LSB) technique.

For TXT Files:

- The message is appended at the end of the file using predefined markers.

For DOCX Files:

- The hidden message is added as a new paragraph in the document.

For PDF Files:

- The original content is extracted and rewritten.
- The hidden message is embedded using very small font size, making it nearly invisible.
- The encoded file is generated and provided for download.

2. Detection Phase

In this phase, the system checks whether a file contains hidden data:

- The user uploads a file for analysis.
- The system identifies the file type.
- The system performs detection based on file type.

For Image Files:

- Extract least significant bits from pixel values.
- Reconstruct the hidden data.
- Check for predefined markers.

For TXT, DOCX, PDF Files:

- Extract file content.
- Search for marker patterns indicating hidden data.

Based on the analysis, the system outputs:

- "Hidden Data Detected"
- "No Hidden Data"

2.3 Algorithm:

Step 1: Initialization:

1. Import required libraries (Streamlit, NumPy, PIL, etc.).
2. Define marker strings for hidden message identification.
3. Initialize user interface components.

Step 2: Encoding Process

1. User selects Encode mode.
2. Upload file and enter secret message.
3. Identify file type:
    - If image → apply LSB encoding.
    - If TXT → append message with markers.
    - If DOCX → insert message as paragraph.
    - If PDF → embed message invisibly.
4. Save encoded file.
5. Provide download option.

Step 3: Detection Process

1. User selects Detect mode.
2. Upload file.
3. Identify file type:
    - If image → extract LSB bits and reconstruct message.
    - If text-based file → extract content.
4. Search for marker string.
5. Determine result:
    - If marker found → Hidden Data Detected
    - Else → No Hidden Data

Step 4: Result Display

1. Display result to user:
    - "Hidden Data Detected"
    - "No Hidden Data"
2. Ensure clear and user-friendly output through interface.

3. IMPLEMENTATION

Source code:

encode.py

```
from PIL import Image

import numpy as np

from docx import Document

from PyPDF2 import PdfReader

from reportlab.pdfgen import canvas

from reportlab.lib.pagesizes import letter

MARKER_START = "<<<HIDDEN_START>>>"

MARKER_END = "<<<HIDDEN_END>>>"

# ---------- IMAGE ----------

def encode_image(image, message):

    img = image.convert('RGB')

    data = np.array(img)

    message = MARKER_START + message + MARKER_END

    binary = ''.join(format(ord(c), '08b') for c in message)

    flat = data.flatten()

    for i in range(len(binary)):

        flat[i] = (flat[i] & 254) | int(binary[i])
```

```
    encoded = flat.reshape(data.shape)

    return Image.fromarray(encoded)


# ---------- TXT ----------

def encode_txt(file, message):

    content = file.read().decode("utf-8")

    hidden = content + "\n" + MARKER_START + message + MARKER_END

    return hidden


# ---------- DOCX ----------

def encode_docx(file, message):

    doc = Document(file)

    hidden = MARKER_START + message + MARKER_END

    doc.add_paragraph(hidden)

    return doc


# ---------- PDF ----------

def encode_pdf(file, message, output_name):

    reader = PdfReader(file)
```

```
text = ""
for page in reader.pages:
    extracted = page.extract_text()
    if extracted:
        text += extracted + "\n"
hidden = MARKER_START + message + MARKER_END
c = canvas.Canvas(output_name, pagesize=letter)
y = 750
for line in text.split("\n"):
    c.drawString(40, y, line[:100])
    y -= 15
    if y < 50:
        c.showPage()
        y = 750
# invisible hidden message
c.setFont("Helvetica", 1)
c.drawString(1, 1, hidden)
c.save()
return output_name
```

detect.py

```
from PIL import Image

import numpy as np

from docx import Document

from PyPDF2 import PdfReader

MARKER_START = "<<<HIDDEN_START>>>"

# ---------- IMAGE ----------

def detect_image(image):

    img = Image.open(image).convert('RGB')

    data = np.array(img)

    flat = data.flatten()

    bits = flat & 1

    chars = []

    for i in range(0, len(bits), 8):

        byte = bits[i:i+8]

        value = int(''.join(map(str, byte)), 2)

        chars.append(chr(value))

    message = ''.join(chars)

    if MARKER_START in message:
```

```
        return "Hidden Data Detected"
    else:
        return "No Hidden Data"
# ---------- TXT ----------
def detect_txt(file):
    content = file.read().decode("utf-8")
    if MARKER_START in content:
        return "Hidden Data Detected"
    else:
        return "No Hidden Data"
# ---------- DOCX ----------
def detect_docx(file):
    doc = Document(file)
    text = " "
    for para in doc.paragraphs:
        text += para.text
    if MARKER_START in text:
        return "Hidden Data Detected"
    else:
```

```
        return "No Hidden Data"

# ---------- PDF ----------

def detect_pdf(file):

    reader = PdfReader(file)

    text = " "

    for page in reader.pages:

        extracted = page.extract_text()

        if extracted:

            text += extracted

    if MARKER_START in text:

        return "Hidden Data Detected"

    else:

        return "No Hidden Data"
```

steg.py

```
import streamlit as st

from encode import *

from detect import *

from PIL import Image

import os

st.title("Multi-format Steganalysis Tool")

mode = st.sidebar.selectbox("Mode", ["Encode", "Detect"])

uploaded_file = st.file_uploader(

    "Upload file",

    type=["png","jpg","jpeg","txt","docx","pdf"]

)

st.markdown("""

<div class="header-box">

    <div class="header-title">

        Multi-format Steganalysis Tool

    </div>

</div>

" " ", unsafe_allow_html=True)
```

```
# ---------------- ENCODE ----------------

if mode == "Encode":

    message = st.text_area("Enter secret message")

    if uploaded_file and message:

        filename = uploaded_file.name

        name, ext = os.path.splitext(filename)

        output_name = f"{name}_enc{ext}"

        if st.button("Encode"):

            # ------------IMAGE-----------------

            if ext.lower() in [".png",".jpg",".jpeg"]:

                image = Image.open(uploaded_file

                # convert JPG to PNG internally

                encoded = encode_image(image, message

                output_name = f"{name}_enc.png

                encoded.save(output_name

                with open(output_name,"rb") as f:

                    st.download_button("Download", f, output_name

            # -------------TXT------------------------

            elif ext == ".txt":
```

```
            encoded = encode_txt(uploaded_file, message)
            st.download_button(
                "Download",
                encoded,
                output_name
            )
        # ----------------DOCX------------------
        elif ext == ".docx":
            doc = encode_docx(uploaded_file, message)
            doc.save(output_name)
            with open(output_name,"rb") as f:
                st.download_button("Download", f, output_name)
        # -----------------PDF--------------------
        elif ext == ".pdf":
            encode_pdf(uploaded_file, message, output_name)
           with open(output_name,"rb") as f:
                st.download_button("Download", f, output_name)
# ---------------- DETECT ----------------
if mode == "Detect":
```

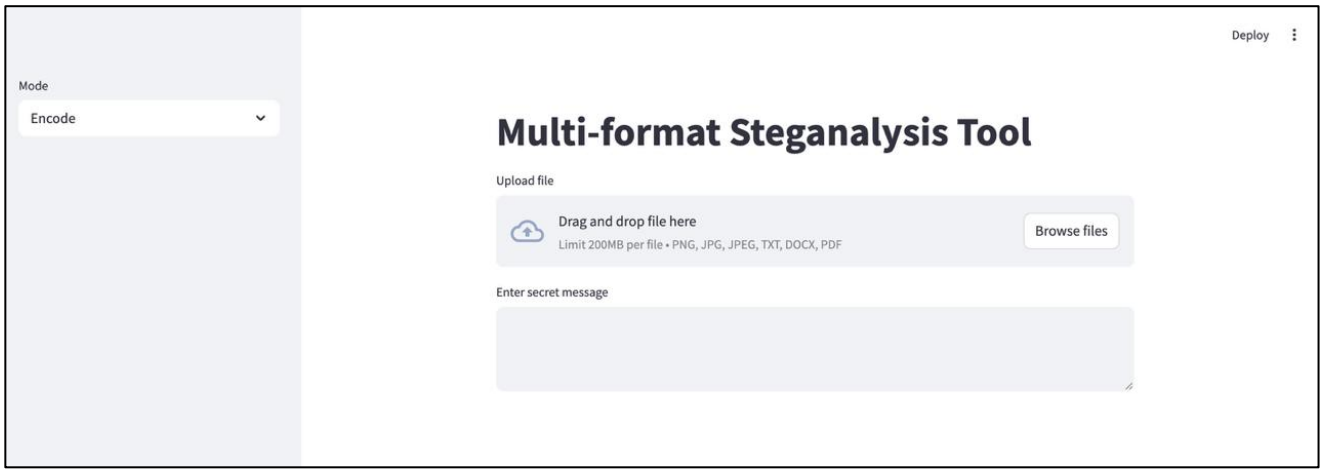
```
if uploaded_file:
    ext = uploaded_file.name.split(".")[-1]
    if st.button("Detect"):
        if ext in ["png","jpg","jpeg"]:
            result = detect_image(uploaded_file)
        elif ext == "txt":
            result = detect_txt(uploaded_file)
        elif ext == "docx":
            result = detect_docx(uploaded_file)
        elif ext == "pdf":
            result = detect_pdf(uploaded_file)
        st.success(result)
```

4. RESULTS

Output Screens:

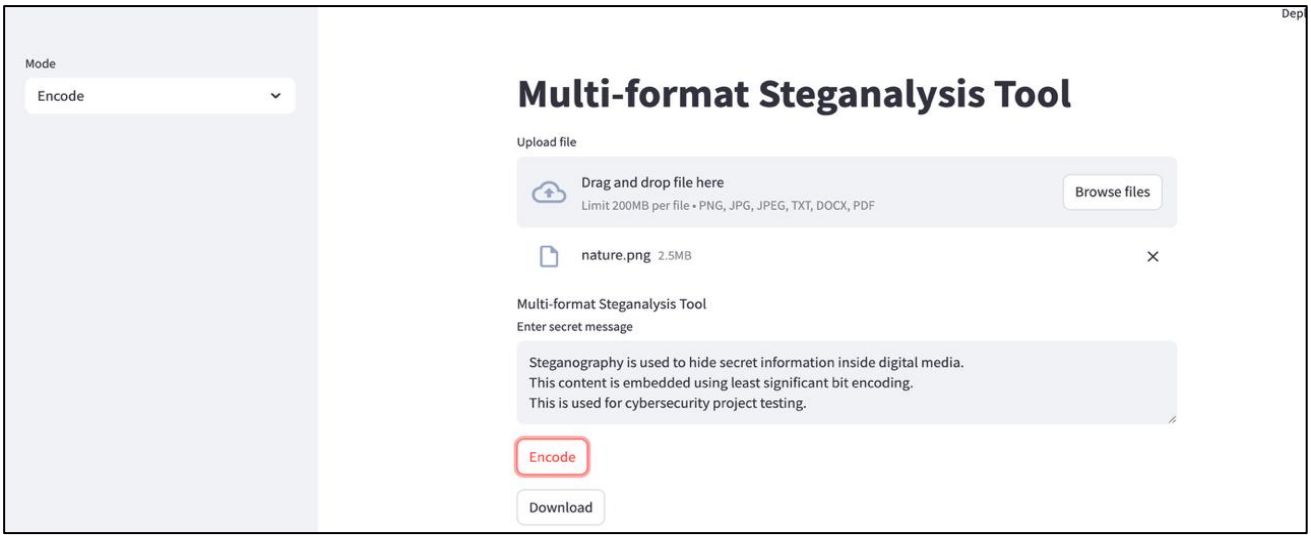
1. Home Screen (Encode Mode – Initial View)

This screen shows the main interface of the application with the **Encode mode selected**. The user can upload a file and enter a secret message. It provides a simple and user-friendly layout for starting the encoding process.



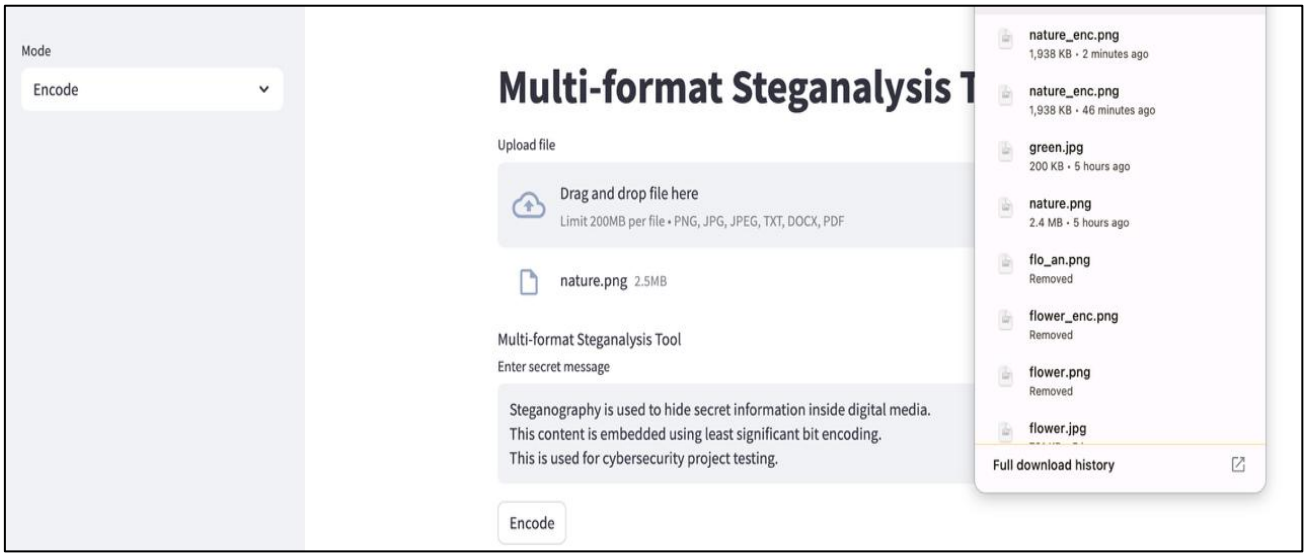
2. File Uploaded & Message Entered

In this screen, the user has successfully uploaded an image file **"nature.png"** and entered a secret message. The system is now ready to perform the encoding operation when the user clicks the encode button.



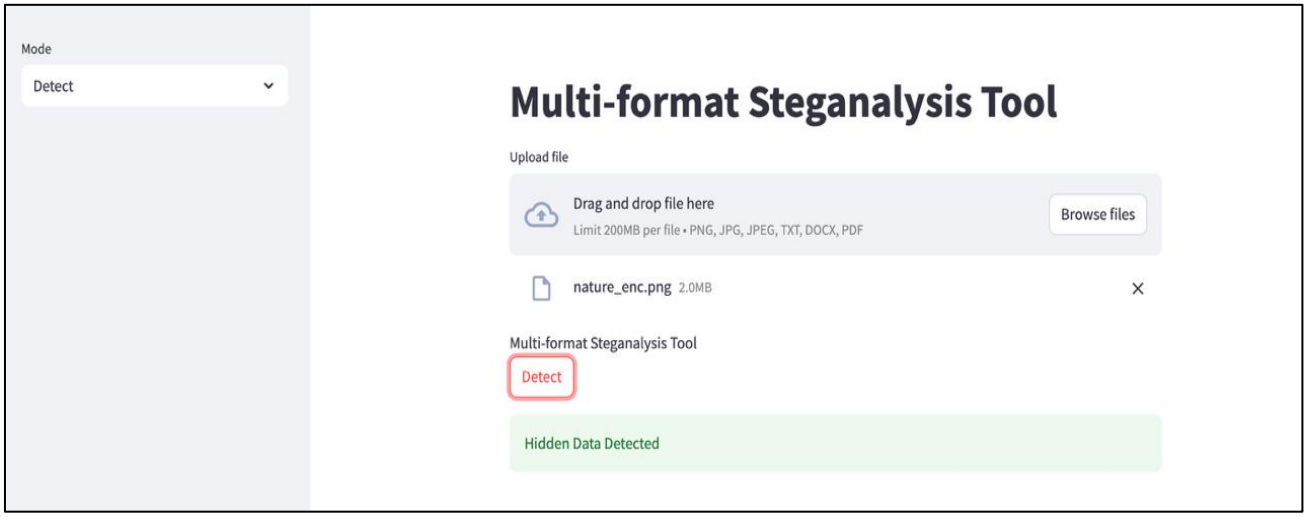
3. Encoding Process & Download Option

After clicking the encode button, the system embeds the secret message into the file. A **download option** is provided to save the encoded file with the modified content. The encrypted file is downloaded with **"filename_enc.filetype"**(nature_enc.png).



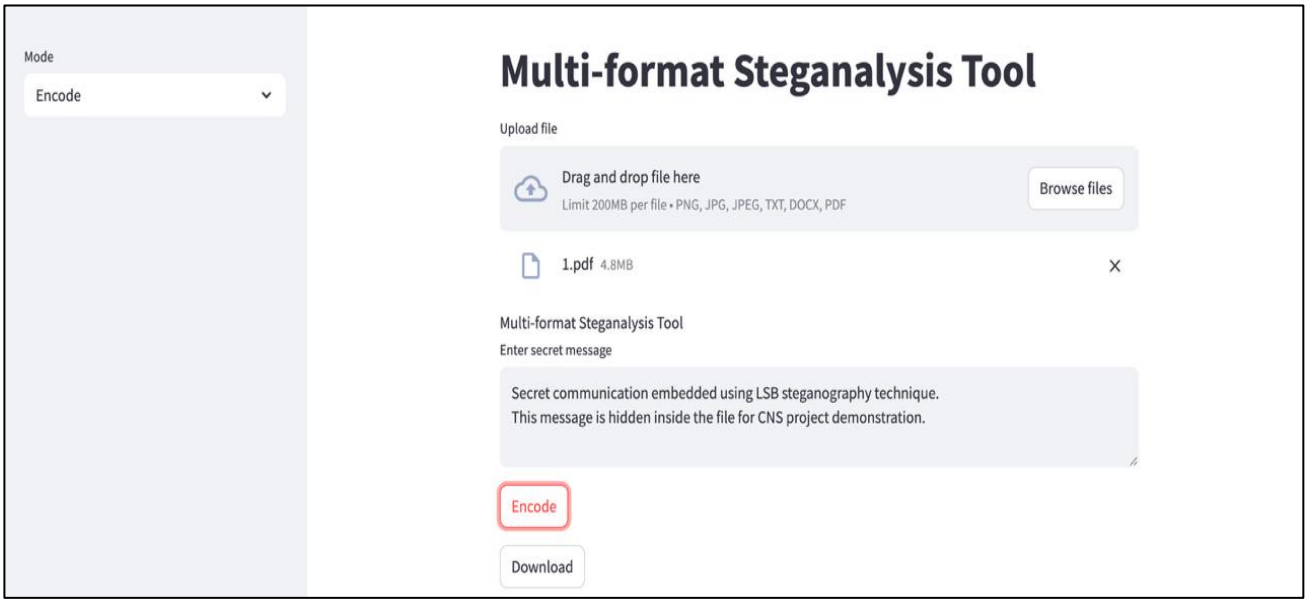
4. Detection Mode – Encoded File

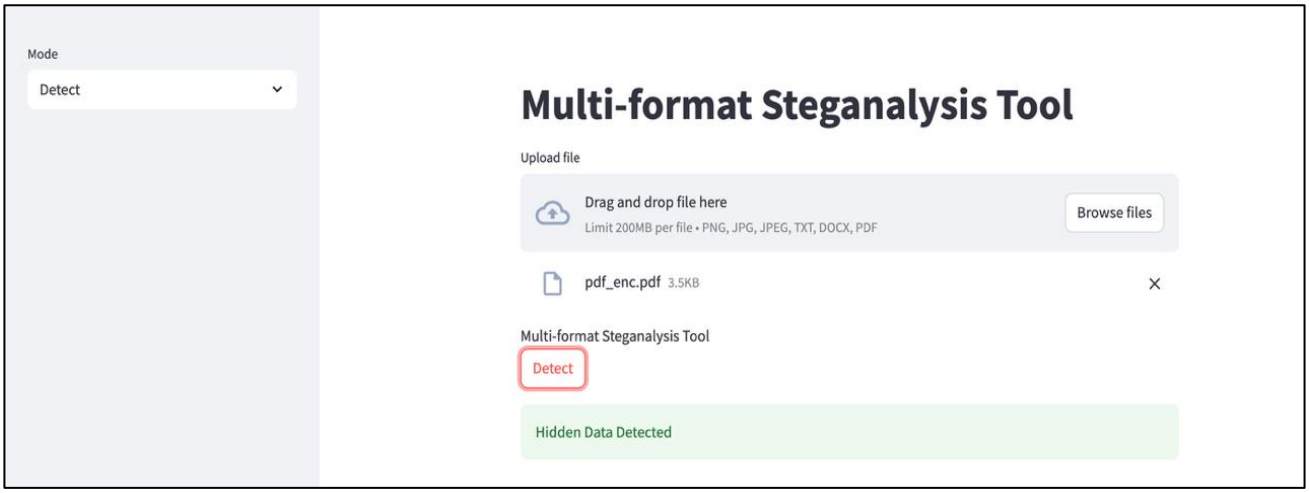
In this screen, the user switches to **Detect mode** and uploads the encoded file. The system prepares to analyze the file for hidden data. After clicking the detect button, the result is displayed as **"Hidden Data Detected"**, confirming successful steganography.



5. Encoding with Different File (PDF Example)

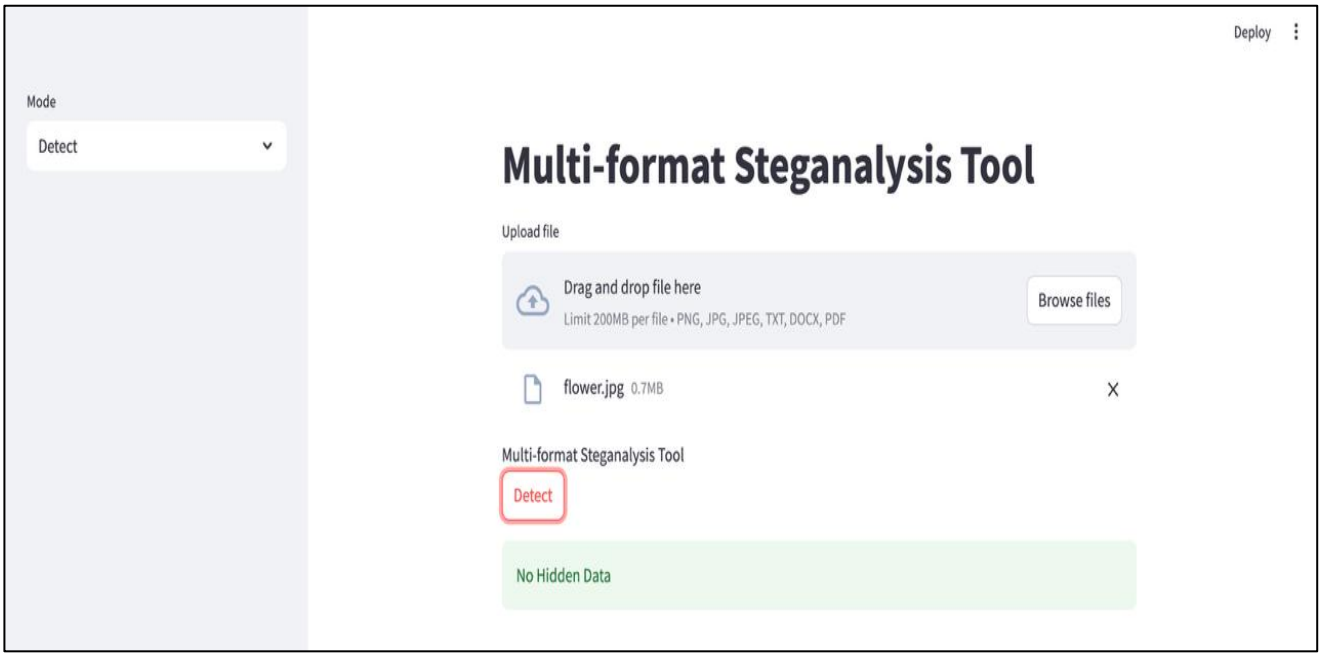
This screen demonstrates encoding in another format (PDF). The system supports multiple file types, showing its flexibility and multi-format capability.





6. No Hidden Data (Normal File)

This screen shows the detection result for a normal file without any hidden message. The system correctly displays **"No Hidden Data"**, indicating accurate detection functionality.



5. CONCLUSION

The **Multi-format Steganalysis System** successfully demonstrates the implementation of both steganography and steganalysis techniques across multiple digital file formats. The primary objective of the project, which is to securely hide confidential information within files and detect the presence of such hidden data, has been effectively achieved. The system supports various formats including images (PNG, JPG, JPEG), text files (TXT), Word documents (DOCX), and PDF files, making it versatile and suitable for real-world applications.

The tool utilizes the **Least Significant Bit (LSB) technique** for image-based data hiding, ensuring that the visual quality of the image remains unaffected while embedding secret information. For text-based files, a **marker-based approach** is implemented, which allows efficient insertion and retrieval of hidden messages without altering the visible structure of the content. The detection module accurately identifies the presence of hidden data by analyzing file content and searching for predefined markers or encoded patterns.

The system is developed using Python and integrates multiple libraries such as Streamlit, NumPy, Pillow, python-docx, PyPDF2, and ReportLab, which collectively contribute to efficient processing and a user-friendly interface. The application is simple to use, provides quick results, and ensures smooth interaction between the user and the system.

Although the project achieves its intended goals, it has certain limitations. The absence of advanced encryption techniques means that the hidden data is not fully secure against sophisticated attacks. Additionally, the detection mechanism relies on predefined markers, which may not be effective for detecting more complex or unknown steganographic methods.

In conclusion, the project provides a strong foundation for understanding and implementing steganography and steganalysis techniques. It has significant applications in cybersecurity, digital forensics, and secure communication. Future enhancements can include the integration of encryption algorithms, advanced detection methods, and support for additional multimedia formats such as audio and video, thereby improving the overall security, efficiency, and robustness of the system.

6. FUTURE SCOPE

The proposed **Multi-format Steganalysis System** provides a strong foundation for secure data hiding and detection; however, it can be further enhanced in several ways to improve functionality, efficiency, and real-world applicability.

- **Advanced Steganography Techniques**
  The current system uses basic LSB and marker-based methods. In the future, more advanced techniques such as frequency-domain embedding (DCT, DWT) can be implemented to increase security and make hidden data more resistant to detection.

- **Machine Learning-based Steganalysis:**
  Detection can be improved by integrating machine learning models to identify hidden patterns instead of relying only on marker-based detection. This would allow detection of unknown or more sophisticated hidden data.

- **Support for Additional File Formats:**
  The system can be extended to support more formats such as audio (MP3, WAV) and video (MP4), making it a more comprehensive steganalysis tool.

- **Encryption Integration:**
  Before embedding, the secret message can be encrypted using cryptographic algorithms such as AES or RSA to enhance data security.

- **Real-time Monitoring System:**
  The tool can be developed into a system that scans files in real-time to detect hidden data in emails, uploads, or network transfers for cybersecurity applications.

- **Cloud-based Deployment:**
  The application can be deployed on cloud platforms to allow remote access and scalability, enabling multiple users to use the system simultaneously.

- **Improved User Interface:**
  Enhancements in UI/UX can be made to provide better visualization, drag-and-drop functionality, and detailed analysis reports.

- **Performance Optimization:**
  Optimization techniques can be applied to improve processing speed, especially for large files such as high-resolution images and lengthy PDFs.

7. REFERENCES

- LSB Image Steganography using Least Significant Bit Algorithm
https://ijariie.com/AdminUploadPdf/Image_Steganography_Using_Least_Significant_Algorithm_ijariie23969.pdf

- Image Steganography using Least Significant Bit (LSB) – Systematic Literature Review
https://www.researchgate.net/publication/358680915_Image_Steganography_using_Least_Significant_Bit_LSB_-_A_Systematic_Literature_Review

- Steganalysis: Detecting LSB Techniques
https://arxiv.org/pdf/1405.5119

- LSB Based Image Steganography using Random Pixel and Bit Selection
https://www.researchgate.net/publication/353598043_A_LSB_Based_Image_Steganography_Using_Random_Pixel_and_Bit_Selection_for_High_Payload

- Pillow (PIL) Documentation
https://pillow.readthedocs.io/

- ReportLab Documentation
https://www.reportlab.com/docs/reportlab-userguide.pdf

- GeeksforGeeks – Image Processing in Python
https://www.geeksforgeeks.org/python-pillow-a-fork-of-pil/